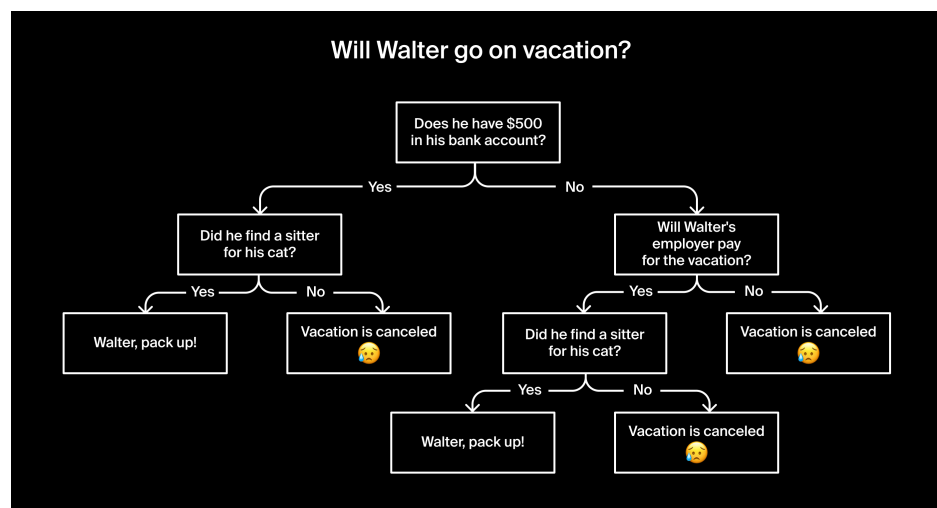


Choosing ML Algorithm

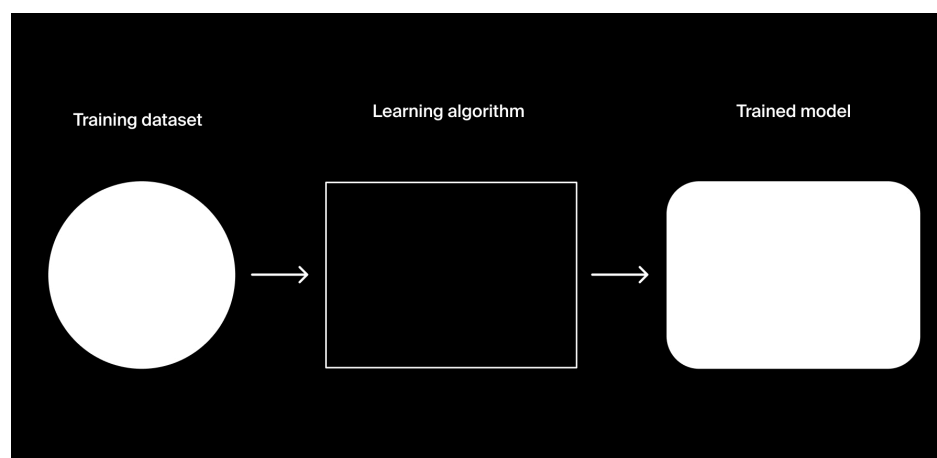
Models and Algorithms

There are many different models that can be used to represent how the features get transformed into the target, each model entailing its own assumptions about how this relationship is structured.

One popular model is called a **decision tree**. It can describe the decision-making process in almost any situation. That's how we make a decision tree with yes/no answers and different scenarios.

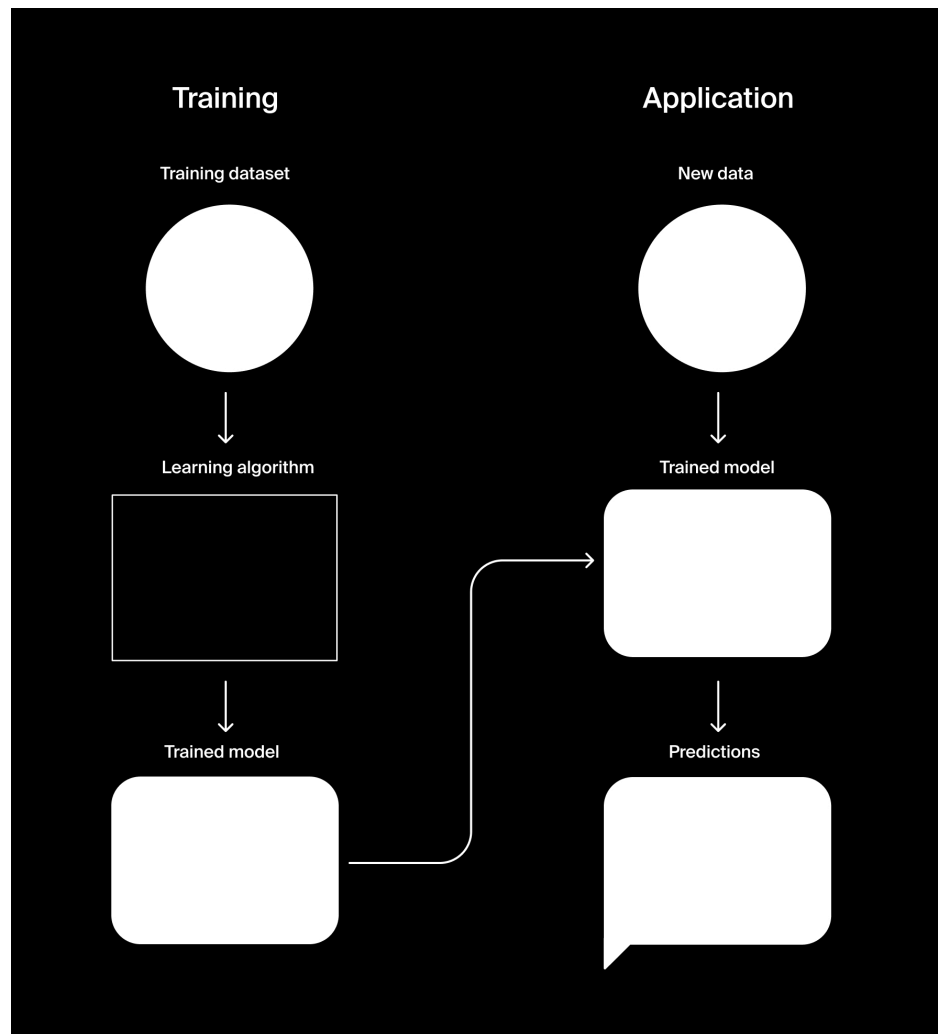


Each tree comes out differently. We will **train the model** to build the most suitable one. In addition to the dataset, we'll need a **learning algorithm**. The dataset is processed through our learning algorithm, producing a **trained model**.



After training, the model is ready to make **predictions**.

It's important to remember that the machine learning process has two steps: model training and model application.



Scikit-learn Library

Sklearn is a great source of tools for working with data and models. For convenience, the library is split into modules. Decision trees are stored in the **tree** module.

Every model corresponds to a separate class in *sklearn*. **DecisionTreeClassifier** is a class for decision tree classifications

COPY PYTHON

```
1 from sklearn.tree import DecisionTreeClassifier
```

Then we create an instance of the class:

COPY PYTHON

```
1 model = DecisionTreeClassifier()
```

The `model` variable now stores our model, and we have to run a learning algorithm to train the model to make predictions.

To initiate training, call the `fit()` method and pass it our variables as an argument.

[COPY PYTHON](#)

```
1 model.fit(features, target)
```

Now we have a trained model in the `model` variable. To predict answers, call the `predict()` method and pass it the table with the features of the new observations.

[COPY PYTHON](#)

```
1 answer = model.predict(new_features)
```

Randomness in learning algorithms

Shuffling introduces randomness to the learning algorithm. A computer can't generate truly random numbers. It uses **Pseudorandom Number Generators** that create sequences that appear random.

All you have to do to add pseudo-randomness when creating a learning algorithm is specify the `random_state` parameter:

[COPY PYTHON](#)

```
1 # specify a random state (number)
2 model = DecisionTreeClassifier(random_state=54321)
3
4 # train the model the same way as before
5 model.fit(features, target)
```

If you set `random_state` to `None` (its default), the pseudorandomness will always be different.

Hyperparameters

Hyperparameters are settings for learning algorithms. You need to specify them before training.

For example, in the decision tree, hyperparameters are:

- `max_depth` — the max depth of the tree.
- `criterion` — the criterion of splitting.
- `min_samples_split` — this prohibits creating nodes that don't contain enough observations from the training set.
- `min_samples_leaf` — leaves are the lowest nodes with the answers that do not split the data any further.

Algorithms

Classification Task

Decision Tree

COPY PYTHON

```
1 from sklearn.tree import DecisionTreeClassifier
2
3 model = DecisionTreeClassifier()
```

Random Forest

A random forest helps to improve results and avoid overfitting.

In the scikit-learn library, you can find the `RandomForestClassifier`, which is a random forest algorithm. Import it from the **ensemble** module:

COPY PYTHON

```
1 from sklearn.ensemble import RandomForestClassifier
```

To set the number of trees in the forest, we will use the `n_estimators` (*number of estimators*) hyperparameter. The quality of the end result and the duration of training are directly proportional to the number of trees.

COPY PYTHON

```
1 model = RandomForestClassifier(random_state=54321, n_estimators=3)
```

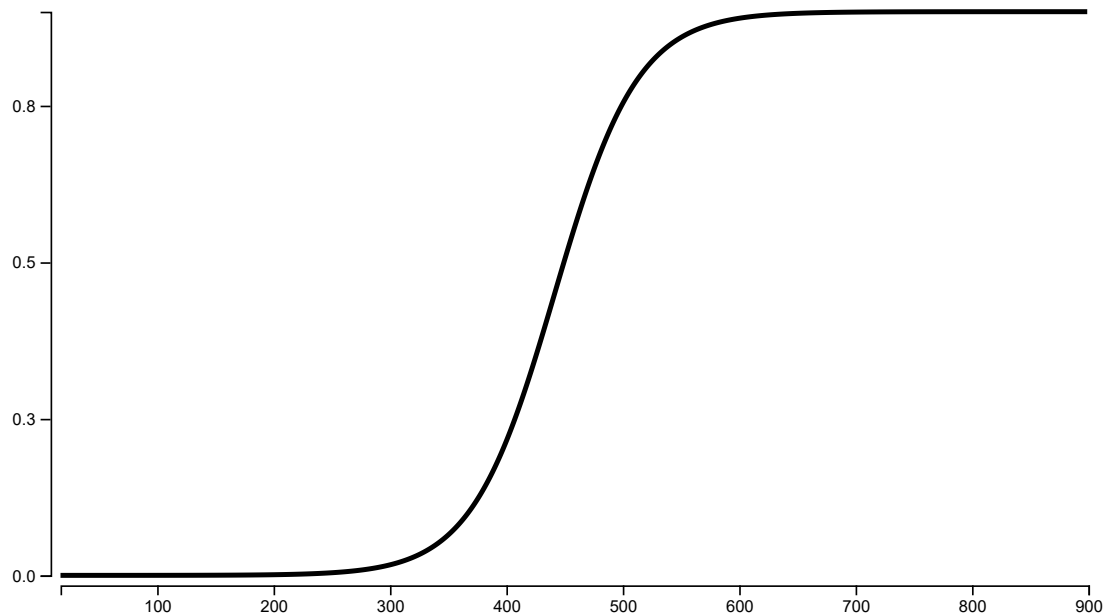
Logistic Regression

To predict the class of an apartment, logistic regression does the following:

- First, it decides which class the observation is closest to.
- Depending on the answer, it chooses the class: if the calculation result is positive, then "1" (high prices); negative — "0" (low prices).

Add points to the graph below to see this in action!

Logistic Regression



There are only a few parameters in logistic regression. The model will not be able to memorize anything from the features in the formula, so the probability of overfitting is low.

The `LogisticRegression` model is located in `sklearn.linear_model` module of the `sklearn` library.

[COPY PYTHON](#)

```
1 from sklearn.linear_model import LogisticRegression
2
3 model = LogisticRegression(random_state=54321)
```

Regression Task

Linear Regression

Linear regression is similar to logistic regression in several ways. The name comes from linear algebra. Linear regression is less susceptible to overfitting because it doesn't have many parameters.

COPY PYTHON

```
1 from sklearn.linear_model import LinearRegression
2
3 model = LinearRegression()
```

Decision Tree Regressor

For regression tasks, decision trees are trained in a manner similar to classification, but they predict a number, not a class.

The decision tree for regression tasks is called `DecisionTreeRegressor` and is located in the `sklearn.tree` module.

COPY PYTHON

```
1 from sklearn.tree import DecisionTreeRegressor
2
3 model = DecisionTreeRegressor(random_state=54321)
```

Random Forest Regressor

COPY PYTHON

```
1 # Initializing the random forest model for regression
2
3 from sklearn.ensemble import RandomForestRegressor
4
5 model = RandomForestRegressor(random_state=54321, n_estimators=3)
```

Tools

Joblib

To save the trained model in the correct format, use the *dump* function of this library.

COPY PYTHON

```
1 # save model
2 # first argument is model
3 # second argument is path to file
4
5 from joblib import dump
6
7 joblib.dump(model, 'model.joblib')
```

You can open and run the model using the *load* function.

```
1 import joblib
```

```
2  
3 # an argument is a path to the file  
4 # a return value is the model  
5 model = joblib.load('model.joblib')
```